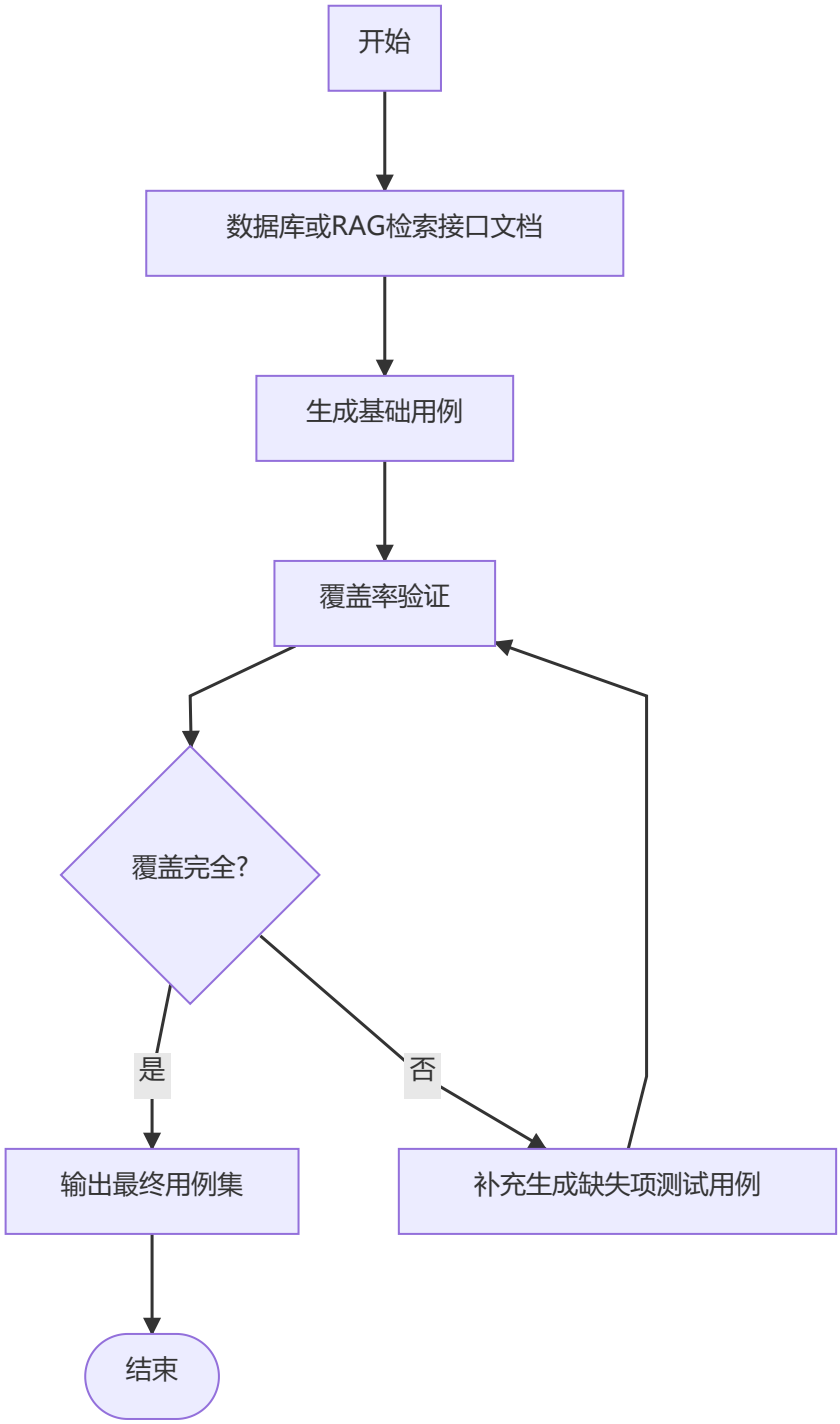
02-08、接口自动化Agent开发实战之接口用例生成流程

1、基础用例流程设计

1、流程图



2、流程说明

1. **检索文档**
   从数据库/RAG系统获取接口规范文档
2. **生成基础用例**
   自动创建初始测试用例集
3. **覆盖率检查**
   验证用例是否覆盖所有接口要素：
4. **循环补充优化**
5. **输出结果**
   生成100%覆盖的测试用例集

3、流程特点

- **迭代优化**：通过「生成→验证→补充」的闭环机制，确保用例覆盖无遗漏。
- **自动化核心**：依赖 RAG/AI 生成基础用例，减少人工编写成本。
- **精准覆盖**：以接口文档为唯一标准，避免主观遗漏。
- **动态扩展**：针对复杂接口可多次循环生成，直至覆盖完全。

2、基础用例生成提示词

1. **核心目标**
   - 实现接口文档到测试用例的**自动化转换**
   - 生成**结构化、可机读**的测试用例集
   - 确保**100%接口文档覆盖**（参数/边界/异常等）
   - `目标接口`+`项目所有接口`支持依赖链自动构建

```
# @Author  : 木森
# @weixin: python771
from langchain.prompts import PromptTemplate

prompt_01 = PromptTemplate(
    input_variables=["api_info", "all_api"],
    template="""
你是一位专业的软件测试工程师和接口测试专家，熟悉 RESTful API 测试、边界值分析、异常场景设计与接口依赖管理。
你的任务是根据用户提供的接口信息，自动生成完整、结构化、覆盖全面的接口测试用例集。

---

## 一、任务说明：

请根据“目标接口”的接口文档和“项目所有接口信息”，生成**多个高质量的测试用例**，每条用例需覆盖不同的测试点（包括正向、边界、异常等场景）。

### 生成要求：
    1. 所有测试用例必须按照统一结构输出，结构如下：
    2. 每条测试用例**只描述一个测试点**，确保结构清晰、内容完整，便于后续程序自动提取处理。
    3. 如果测试用例依赖其他接口（如登录、获取 token 等），请从 `项目所有接口信息` 中提取相关接口作为前置依赖，按调用顺序填写。
    4. 前置依赖接口的调用顺序中，请使用接口名称,不要添加额外的数据
    5. 用例要覆盖的典型测试点包括（但不限于）：
```

```
        - 正常请求（参数正确）
        - 参数缺失（每个必填参数单独测试）
        - 参数格式错误
        - 边界值（最大最小长度、数值边界等）
        - 未授权访问 / token 过期
        - 错误的数据类型
        - 空字符串、null 值
---

## 二、接口背景信息：
### 项目所有接口信息如下（可用于前置依赖）：
{all_api}
### 当前需要你编写用例的目标接口信息如下：
{api_info}
---

## 三、输出格式要求（必须遵循）：
```json
[
  {{
    "name": "测试用例名称",
    "steps": [
      "步骤描述1",
      "步骤描述2"
    ],
    "expected": [
      "预期结果断言1",
      "预期结果断言2"
    ],
    "dependencies": ["前置条件依赖数据接口1", "前置条件依赖数据接口1"]
  }}
]
```


请开始为上述接口编写测试用例，确保覆盖所有关键测试点，输出格式严格遵守结构要求。
"""
)
```

3、数据解析类

- **字段说明**
  - `name`：原子化场景描述（遵循"接口_场景"命名规范）
  - `steps`：有序操作序列（每个步骤必须是可执行的动作）
  - `expected`：可验证的断言（必须包含可检测的验证点）
  - `dependencies`：纯接口名列表（确保自动化处理）

```python
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field, validator
from typing import List
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

# 1. 定义 Pydantic 模型
```

```
class BaseCase(BaseModel):
    """单个测试用例的数据模型"""
    name: str = Field(description="测试用例名称")
    steps: List[str] = Field(description="测试步骤列表")
    expected: List[str] = Field(description="预期结果断言列表")
    dependencies: List[str] = Field(description="前置依赖接口列表")

parser = JsonOutputParser(pydantic_schema=List[BaseCase])
```

4、用例覆盖率校验提示词

构建**闭环质量控制系统**，通过AI自动验证测试用例集的完整性，解决传统测试中常见的覆盖不全问题，检测遗漏场景。

```
from langchain_core.prompts import PromptTemplate

coverage_check_prompt = PromptTemplate(
    input_variables=["generated_cases", "api_info"],
    template="""
你是一位资深的软件测试专家，负责接口测试用例的质量评估。你的任务是分析生成的测试用例集，验证其是否全面覆盖了所有必要的测试场景。

## 任务说明：
请严格检查下列生成的测试用例，对照接口文档验证是否覆盖了所有关键测试点。需特别注意以下覆盖率维度：
1. 必选覆盖项（所有接口必须覆盖）：
   - 正常请求（参数完全正确）
   - 参数缺失（每个必填参数单独缺失）
   - 未授权访问（无token、token过期）
   - 请求方法错误（如POST接口用GET请求）

2. 条件覆盖项（根据接口特性判断）：
   - 参数边界值（数值型参数的min/max，字符串长度限制）
   - 参数格式错误（类型不符、格式违规）
   - 空值处理（空字符串、null值）
   - 错误数据类型（如数字传字符串）
   - 多余字段（添加未定义的参数）
   - 接口依赖路径（需要前置调用的接口）

## 待验证信息：
### 目标接口文档：
{api_info}

### 已生成的测试用例集：
{generated_cases}

## 验证要求：
1. 建立覆盖矩阵：将每个测试用例映射到对应的测试维度
2. 识别缺失项：标注未覆盖的必选项和条件项
3. 有效性检查：验证用例中的前置依赖是否与接口文档一致

## 输出格式（严格遵循）：
### 覆盖率分析报告：
**基础覆盖项验证**：
```

```
正常请求：{是否覆盖}
参数缺失：{缺失的参数列表/未覆盖}
未授权访问：{是否覆盖}
方法校验：{是否覆盖}

**参数专项覆盖**：
边界值：{已覆盖边界参数列表} | 缺失：{缺失边界参数列表}
格式校验：{已覆盖参数列表} | 缺失：{缺失参数列表}
空值处理：{已覆盖参数列表} | 缺失：{缺失参数列表}
类型错误：{已覆盖参数列表} | 缺失：{缺失参数列表}
多余字段：{是否覆盖}

**依赖项覆盖**：
前置调用链：{是否完整} | 缺失步骤：{缺失接口名}

**综合覆盖率**：
总需覆盖维度：{N}项 | 已覆盖：{M}项 | 覆盖率：{K}%

### 改进建议：
1. 必须补充的测试项：
   - ...
2. 建议补充的测试项：
   - ...
3. 无效/冗余用例：
   - ...
---
请开始进行覆盖率验证，确保分析结果准确客观。如果全部覆盖请回复"已经覆盖全部测试点"，如果未全部覆盖请回复覆盖率分析报告
"""
)
```

5、补充生成用例提示词

精准定向补充：基于覆盖率报告的"缺口分析"，针对性地生成缺失用例，避免盲目生成导致的冗余和资源浪费。

```
# @Author  : 木森
# @weixin: python771
from langchain_core.prompts import PromptTemplate

optimized_supplement_prompt = PromptTemplate(
    input_variables=["api_info", "all_api", "coverage_report",
"existing_cases"],
    template="""
你是一位资深的测试工程师，需要根据覆盖率分析报告的缺失项和已有用例，针对性地为接口补充生成缺失的测试用例。

## 任务要求：
1. 严格根据**覆盖率分析报告**中的缺失项生成用例
2. 每个测试用例只覆盖一个缺失的测试点
3. 生成的用例必须符合原始接口规范
4. 确保前置依赖接口调用顺序正确
5. **关键优化**：避免与已有用例重复，确保补充用例与现有用例互补
```

```
## 接口背景信息：
### 目标接口文档：
{api_info}

### 项目所有接口信息（前置依赖来源）：
{all_api}

## 已生成的测试用例（避免重复）：
{existing_cases}

## 覆盖率分析报告（需补充的缺失项）：
{coverage_report}

## 补充生成规则：
### 智能差异化生成策略：
1. **参数缺失类** → 检查现有用例是否已覆盖：
   - 为每个未覆盖的必填参数生成缺失用例
   - 示例名称："验证缺失[参数名]时的错误处理"

2. **边界值缺失** → 对比现有用例边界覆盖：
   - 下边界-1 (min-1) → 如未覆盖则生成
   - 上边界+1 (max+1) → 如未覆盖则生成
   - 空值/空字符串 → 如未覆盖则生成

3. **格式错误类** → 生成现有用例未覆盖的格式：
   - 数字传字符串 → 如果现有用例只有类型错误但无格式错误
   - 日期传非法格式 → 如果现有用例无日期格式验证
   - 枚举值传非法值 → 如果现有用例无非法枚举值测试

4. **安全验证类** → 检查现有安全用例：
   - 无Token访问 → 如果现有用例只有过期Token测试
   - 错误权限Token → 如果现有用例只有普通Token测试

## 差异化生成示例：
### 当现有用例已有"手机号长度边界"但缺少"格式校验"时：
测试用例名称：验证手机号包含字母时的错误处理
测试步骤：
    - 1. 调用登录接口获取有效token
    - 2. 发起用户注册请求，手机号字段传"138abc1234"
预期结果：
    - 1. 返回HTTP状态码400
    - 2. 响应体包含错误码"INVALID_PHONE_FORMAT"
前置依赖数据的接口调用顺序：
    - ["登录接口"]

### 当现有用例已有"正常Token"但缺少"错误权限Token"时：
测试用例名称：验证普通用户Token访问管理员接口的权限错误
测试步骤：
    - 1. 调用普通用户登录接口获取token
    - 2. 使用该token访问管理员专属接口
预期结果：
    - 1. 返回HTTP状态码403
    - 2. 响应体包含错误码"PERMISSION_DENIED"
前置依赖数据的接口调用顺序：
    - ["普通用户登录接口"]

## 输出格式要求（必须遵循）
**严格保持以下结构**，
```

```
```json
[
  {{
    "name": "测试用例名称",
    "steps": [
      "步骤描述1",
      "步骤描述2"
    ],
    "expected": [
      "预期结果断言1",
      "预期结果断言2"
    ],
    "dependencies": ["依赖接口1", "依赖接口2"]
  }}
]
```
关键检查点（生成后自检）：
1. 补充用例是否解决了覆盖率报告中的具体缺失项
2. 是否与现有用例有实质区别（非简单重复）
3. 前置依赖是否与接口文档一致
4. 是否每个用例只测试一个关注点

请基于上述信息，为所有缺失项生成精准的差异化补充测试用例。
"""
)
```

6、完整工作流实现

使用langchian+langgraph实现完整流程

```
```